

Programación Lineal con Backtracking

Documentación Técnica

Inteligencia Artificial

Índice

1. Descripción del Proyecto	2
2. Programación Lineal Entera	2
3. Algoritmo de Backtracking para PLE	2
3.1. Poda	3
4. Problema de la Carpintería	3
4.1. Formulación	3
4.2. Solución óptima	3
5. Interfaz Gráfica	3

1. Descripción del Proyecto

Dos aplicaciones que usan **Backtracking** para resolver problemas de programación lineal entera (PLE):

1. **BacktrackingPLE**: solucionador genérico de PL entera.
2. **BacktrackingCarpinteriaPLE**: problema específico de optimización de ganancias.

2. Programación Lineal Entera

La PLE busca maximizar o minimizar una función objetivo lineal sujeta a restricciones lineales, con la condición de que las variables sean **enteras no negativas**:

$$\text{Maximizar: } Z = c_1x_1 + c_2x_2 + \dots + c_nx_n$$

$$\text{Sujeto a: } a_{ij}x_j \leq b_i \quad \forall i, \quad x_j \geq 0, x_j \in \mathbb{Z}$$

3. Algoritmo de Backtracking para PLE

El backtracking explora el espacio de soluciones enteras de forma sistemática:

```
1 void backtrack(int[] variables, int indice,
2               double[][] A, double[] b, double[] c) {
3     if (indice == variables.length) {
4         if (cumpleRestricciones(variables, A, b)) {
5             double z = calcularObjetivo(variables, c);
6             if (z > mejorZ) {
7                 mejorZ = z;
8                 mejorSolucion = variables.clone();
9             }
10        }
11        return;
12    }
13
14    // Calcular l mite superior para la variable actual
15    int limSup = calcularLimiteSuperior(indice, A, b);
16
17    for (int v = 0; v <= limSup; v++) {
18        variables[indice] = v;
19        // Poda: si la soluci n parcial ya no es factible
20        if (esParcialmenteFactible(variables, indice,
21                                   A, b)) {
22            backtrack(variables, indice+1, A, b, c);
23        }
24    }
25    variables[indice] = 0; // restaurar
26 }
```

3.1. Poda

La **poda** elimina ramas que no pueden conducir a una solución óptima:

- Si la asignación parcial ya viola alguna restricción, se descarta esa rama.
- Si el límite superior de la función objetivo para la solución parcial es menor al mejor encontrado, se poda.

4. Problema de la Carpintería

4.1. Formulación

Una carpintería fabrica mesas (x_1) y sillas (x_2):

$$\begin{aligned} \text{Maximizar: } Z &= 5x_1 + 4x_2 \quad (\text{ganancia en \$}) \\ \text{Sujeto a: } 6x_1 + 4x_2 &\leq 24 \quad (\text{madera disponible}) \\ x_1 + 2x_2 &\leq 6 \quad (\text{horas de trabajo}) \\ x_1, x_2 &\geq 0, \quad x_1, x_2 \in Z \end{aligned}$$

4.2. Solución óptima

El backtracking encuentra $x_1 = 3$, $x_2 = \frac{3}{2}$, pero al requerir enteros: $x_1 = 3$, $x_2 = 1$ con $Z = 19$.

5. Interfaz Gráfica

- Formularios para ingresar coeficientes de la función objetivo y restricciones.
- Tabla de restricciones dinámica (se pueden agregar/quitar filas).
- Botón **Resolver** ejecuta el backtracking y muestra la solución óptima.
- Muestra los valores de todas las variables y el valor óptimo de Z .